

## ОТЧЁТ ПО ЛАБОРАТОРНОЙ РАБОТЕ № 21

Построение интерфейса веб-приложения на Vue.js с использованием  
экземпляра Vue, компонентов и директив  
*Вариант 2 – лендинг IT-конференции*

Выполнила: Клычева Г.

Группа: 310102

Репозиторий: <https://github.com/karrinkl/devconf-vue.git>

Проверил: Прудник М.С.

### 1 ЦЕЛЬ РАБОТЫ И УСЛОВИЕ ЗАДАНИЯ

Цель работы состоит в получении практических навыков создания интерактивных веб-приложений на Vue.js. Требуется реализовать UI-компоненты, спроектированные в лабораторной работе № 17, в виде однофайловых компонентов Vue и освоить систему реактивности, декларативное управление DOM через директивы и обмен данными между компонентами.

По варианту 2 предметной областью выступает лендинг IT-конференции. Обязательный набор включает три компонента: SpeakerCard.vue, SpeakersList.vue и RegistrationForm.vue. В карточке спикера требуется реализовать переключение видимости детальной информации по клику через директиву v-if, в форме регистрации использовать v-model для полей ввода, а по нажатию кнопки генерировать событие с данными формы.

Внешние библиотеки управления состоянием условием задания запрещены, поэтому все данные хранятся в опции data() компонентов и передаются вниз через props, а вверх - пользовательскими событиями.

### 2 СТРУКТУРА ПРОЕКТА

Проект создан на базе сборщика Vite по шаблону Vue. Каждый компонент представляет собой однофайловый компонент с расширением .vue, в котором объединены шаблон, логика и стили. Состав исходных файлов приведён в таблице 1.

Таблица 1 – Структура каталога src

| Путь                            | Назначение                                                                     |
|---------------------------------|--------------------------------------------------------------------------------|
| App.vue                         | Корневой компонент: первый экран, список спикеров, форма и перечень участников |
| main.js                         | Точка входа, создание и монтирование экземпляра приложения                     |
| components/SpeakerCard.vue      | Карточка одного спикера с раскрывающимися подробностями                        |
| components/SpeakersList.vue     | Секция со списком спикеров                                                     |
| components/RegistrationForm.vue | Форма регистрации участника                                                    |
| assets/styles.css               | Глобальные стили и токены дизайн-системы из ЛР № 17                            |
| assets/icons/                   | SVG-иконки и логотип конференции                                               |
| assets/images/                  | Изображения спикеров и баннер в формате WebP                                   |

Экземпляр приложения создаётся в файле main.js вызовом `createApp(App).mount('#app')`. Это точка входа, в которой корневой компонент связывается с элементом разметки, имеющим идентификатор `app`.

### 3 РЕАЛИЗОВАННЫЕ КОМПОНЕНТЫ

Приложение состоит из корневого компонента и трёх дочерних. Входные свойства всех компонентов с указанием типов и обязательности приведены в таблице 2.

Таблица 2 – Входные свойства компонентов

| Компонент        | Свойство    | Тип    | Значение по умолчанию              |
|------------------|-------------|--------|------------------------------------|
| SpeakersList     | speakers    | Array  | обязательное                       |
| SpeakersList     | title       | String | 'Спикеры'                          |
| SpeakerCard      | speaker     | Object | обязательное, с функцией validator |
| RegistrationForm | title       | String | 'Регистрация'                      |
| RegistrationForm | submitLabel | String | 'Зарегистрироваться'               |

Свойство `speaker` снабжено функцией `validator`, которая проверяет наличие полей `id`, `name`, `topic` и `photoUrl`. При передаче объекта неверной структуры Vue выведет предупреждение в консоль в режиме разработки.

#### 3.1 App

Корневой компонент. Хранит массив спикеров и массив зарегистрированных участников, выводит первый экран, подключает список

спикеров и форму регистрации. Метод `handleRegister` принимает данные из события формы и добавляет их в массив `registrations`, после чего список участников перерисовывается автоматически благодаря системе реактивности.

Данные спикеров описаны в `data()` корневого компонента и передаются в список привязкой `:speakers="speakers"`. Изображения импортируются как модули, поэтому сборщик подставляет в разметку итоговые пути и включает файлы в сборку.

### 3.2 SpeakersList

Секция со списком спикеров. Получает массив через `props` и перебирает его директивой `v-for` с обязательным ключом `:key="speaker.id"`. Уникальный ключ позволяет Vue сопоставлять элементы списка между перерисовками и переиспользовать узлы DOM вместо полного пересоздания.

Компонент хранит в `data()` массив `expandedIds` - идентификаторы карточек с раскрытыми подробностями. Он пополняется и очищается обработчиком `handleToggle`, который подписан на событие `toggle` дочерней карточки. Вычисляемое свойство `expandedCount` возвращает длину массива и выводится в подзаголовке секции. Вычисляемое свойство пересчитывается только при изменении зависимостей, что отличает его от метода, вызываемого при каждой перерисовке.

### 3.3 SpeakerCard

Карточка одного спикера. Собственное состояние компонента - логическое поле `areDetailsVisible`, переключаемое методом `toggleDetails` по клику. Значение поля управляет директивой `v-if`, которая добавляет или удаляет из DOM блок с биографией и уровнем доклада. Вычисляемое свойство `toggleLabel` возвращает подпись кнопки в зависимости от текущего состояния.

Помимо изменения собственного состояния метод `toggleDetails` генерирует событие `toggle` с объектом, содержащим идентификатор спикера и новое значение видимости. Родительский компонент использует это событие для подсчёта раскрытых карточек. Список генерируемых событий объявлен в опции `emits`, что документирует внешний интерфейс компонента.

Формат участия выводится цепочкой `v-if`, `v-else-if` и `v-else`: офлайн, онлайн либо сообщение о том, что формат уточняется. Три взаимоисключающих состояния описаны одной цепочкой директив без дублирования разметки.

### 3.4 RegistrationForm

Форма регистрации. Поля имени, адреса электронной почты, выпадающего списка формата участия и флажка подписки связаны с данными компонента директивой `v-model`, которая реализует двустороннюю привязку.

Для текстовых полей применён модификатор `v-model.trim`, убирающий пробелы по краям значения.

Метод `validate` проверяет длину имени и соответствие адреса регулярному выражению, записывая тексты ошибок в объект `errors`. Привязка `:class` добавляет полю класс ошибки, а цепочка `v-if` и `v-else` переключает подсказку на сообщение об ошибке. Метод `submit` при успешной проверке генерирует событие `register` с объектом данных и очищает форму методом `reset`.

Отправка формы перехвачена обработчиком `@submit.prevent`, поэтому страница не перезагружается. Сокращённые формы записи `:` и `@` применены вместо полных `v-bind` и `v-on`, как рекомендует руководство по стилю Vue.

## 4 РЕАКТИВНЫЕ ДАННЫЕ, МЕТОДЫ И СОБЫТИЯ

Состояние каждого компонента объявлено в опции `data()` в виде функции, возвращающей объект. Это обязательное требование для компонентов: при каждом создании экземпляра функция вызывается заново, поэтому разные карточки спикеров не разделяют одно и то же состояние. Перечень реактивных данных приведён в таблице 3.

Таблица 3 – Реактивные данные компонентов App и SpeakersList

| Компонент    | Поле          | Начальное значение   | Назначение                           |
|--------------|---------------|----------------------|--------------------------------------|
| App          | speakers      | массив из 4 объектов | Данные докладов конференции          |
| App          | registrations | пустой массив        | Список зарегистрированных участников |
| SpeakersList | expandedIds   | пустой массив        | Идентификаторы раскрытых карточек    |

Массив `speakers` объявлен один раз в корневом компоненте и передаётся вниз, поэтому источник данных остаётся единственным. Состояние двух дочерних компонентов приведено в таблице 4.

Таблица 4 – Реактивные данные компонентов SpeakerCard и RegistrationForm

| Компонент        | Поле              | Начальное значение        | Назначение                      |
|------------------|-------------------|---------------------------|---------------------------------|
| SpeakerCard      | areDetailsVisible | false                     | Видимость блока с подробностями |
| RegistrationForm | name, email       | пустые строки             | Значения текстовых полей        |
| RegistrationForm | format            | 'offline'                 | Выбранный формат участия        |
| RegistrationForm | acceptsNews       | false                     | Состояние флажка подписки       |
| RegistrationForm | errors            | объект с пустыми строками | Тексты ошибок валидации         |

Методы и вычисляемые свойства компонентов сведены в таблице 5.

Таблица 5 – Методы и вычисляемые свойства

| Компонент        | Член компонента  | Назначение                                                      |
|------------------|------------------|-----------------------------------------------------------------|
| App              | handleRegister() | Принимает данные события register и пополняет массив участников |
| App              | formatLabel()    | Переводит код формата участия в подпись для вывода              |
| SpeakersList     | handleToggle()   | Обрабатывает событие toggle, ведёт список раскрытых карточек    |
| SpeakersList     | expandedCount    | Вычисляемое свойство: число раскрытых карточек                  |
| SpeakerCard      | toggleDetails()  | Переключает видимость подробностей и генерирует событие         |
| SpeakerCard      | toggleLabel      | Вычисляемое свойство: подпись кнопки                            |
| RegistrationForm | validate()       | Проверяет имя и адрес, заполняет объект errors                  |
| RegistrationForm | submit()         | Генерирует событие register и очищает форму                     |
| RegistrationForm | reset()          | Возвращает поля формы к начальным значениям                     |

Пользовательские события, объявленные в опции emits, перечислены в таблице 6.

Таблица 6 – Пользовательские события

| Компонент        | Событие  | Передаваемые данные                               |
|------------------|----------|---------------------------------------------------|
| SpeakerCard      | toggle   | Идентификатор спикера и новое состояние видимости |
| RegistrationForm | register | Имя, адрес, формат участия и согласие на рассылку |

Поток данных односторонний: свойства передаются от родителя к потомку, а обратная связь идёт только через события. Дочерний компонент не изменяет полученные props напрямую, поэтому источник истины для каждой величины остаётся единственным.

## 5 ИСПОЛЬЗОВАННЫЕ ДИРЕКТИВЫ

Места применения директив шаблона приведены в таблице 7.

Таблица 7 – Директивы и места их применения

| Директива       | Где применена                                                                          |
|-----------------|----------------------------------------------------------------------------------------|
| v-for           | Список карточек спикеров и список зарегистрированных участников, оба с уникальным :key |
| v-if            | Блок подробностей о спикере, сообщения об ошибках, непустые списки                     |
| v-else-if       | Бейдж формата участия: вариант «Онлайн»                                                |
| v-else          | Бейдж «Формат уточняется», подсказки под полями, пустые состояния списков              |
| v-model         | Текстовые поля формы с модификатором trim, выпадающий список, флажок                   |
| : вместо v-bind | Пути к изображениям, входные свойства компонентов, объектная запись :class             |
| @ вместо v-on   | Обработчики @click на кнопке карточки, @toggle и @register, @submit.prevent на форме   |

Пустые состояния списков обработаны отдельно: если массив спикеров или участников пуст, вместо перечня выводится поясняющий текст. Такое поведение реализовано парой v-if и v-else без дополнительных вычислений.

## 6 РЕЗУЛЬТАТ РАБОТЫ ПРИЛОЖЕНИЯ

Оформление интерфейса повторяет макет ЛР № 17: те же цвета, типографическая шкала, радиусы скругления и тени карточек. Исходное состояние секции спикеров приведено на рисунке 1.

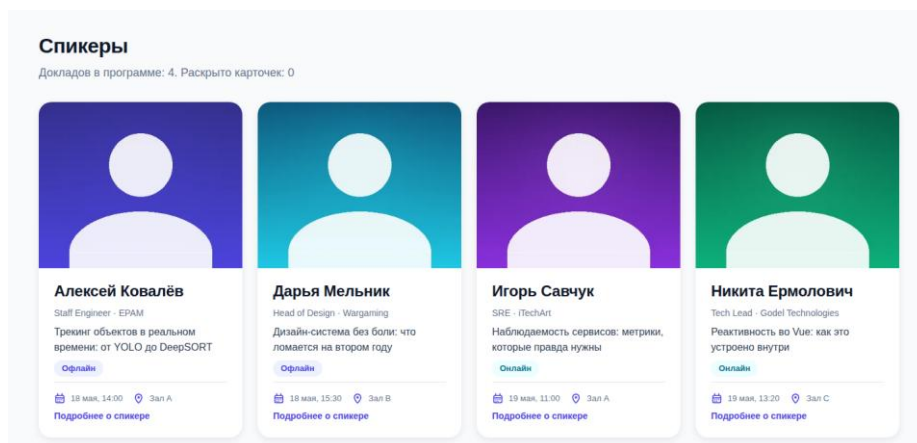


Рисунок 1 – Секция спикеров в исходном состоянии

Все карточки закрыты, счётчик раскрытых карточек в подзаголовке равен нулю. Бейдж формата участия отличается у офлайн- и онлайн-спикеров, что подтверждает работу цепочки v-if, v-else-if и v-else.

Состояние после нажатия на ссылку «Подробнее о спикере» у первой и третьей карточек показано на рисунке 2.

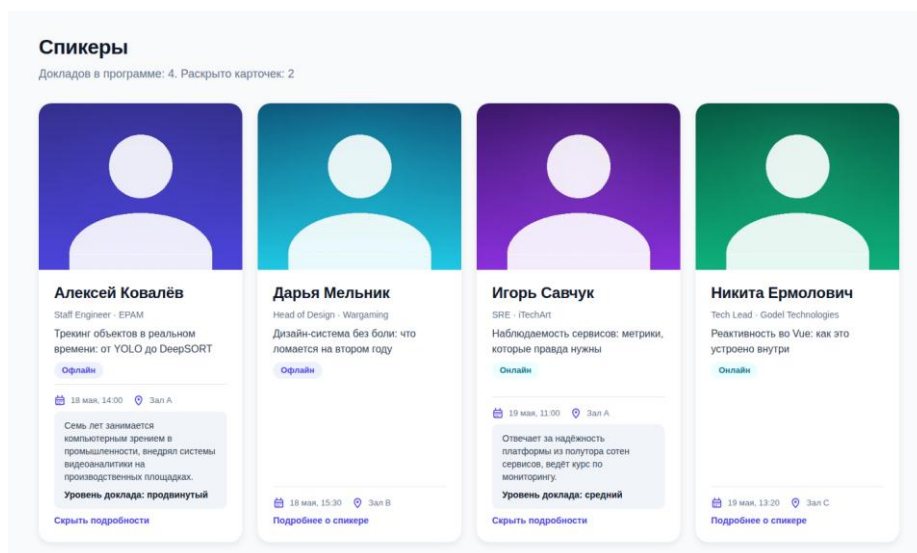


Рисунок 2 – Карточки с раскрытыми подробностями

У раскрытых карточек появился блок с биографией и уровнем доклада, подпись ссылки сменилась на «Скрыть подробности», а счётчик в подзаголовке показывает два. Счётчик обновился в родительском компоненте по событию toggle, то есть данные прошли снизу вверх.

Работа формы регистрации и списка участников показана на рисунке 3.

The image shows a web interface with two main sections. The left section, titled 'Регистрация' (Registration), contains three input fields: 'Имя и фамилия' (Name and Surname) with the value 'Ан', 'E-mail' with the value 'ivan@bsuir', and 'Формат участия' (Participation Format) with a dropdown menu showing 'Онлайн-трансляция'. Below these fields are two error messages: 'Укажите имя и фамилию' (Specify name and surname) and 'Введите корректный e-mail' (Enter a correct email). There is a checkbox labeled 'Присылать анонсы следующих конференций' (Send announcements of the following conferences) which is checked. At the bottom of this section is a blue button labeled 'Зарегистрироваться' (Register) with a right arrow. The right section, titled 'Зарегистрированные участники' (Registered participants), displays a list of two participants. The first participant is 'Иван Иванов' (Ivan Ivanov) with email 'ivan@bsuir.by' and status 'Офлайн' (Offline). The second participant is 'Петр Сидорчук' (Petr Sidorchuk) with email 'petr@example.com' and status 'Онлайн' (Online).

Рисунок 3 – Валидация формы и список зарегистрированных участников

Поля с некорректными значениями подсвечены и снабжены сообщениями об ошибках. Список справа заполнен участниками, добавленными ранее по событию register: корневой компонент принял данные формы и пополнил массив registrations, а директива v-for вывела его на экран.

## 7 ЗАПУСК ПРОЕКТА

Для запуска требуется Node.js версии 18 или новее. Последовательность команд:

- npm install - установка зависимостей;
- npm run dev - запуск сервера разработки, по умолчанию на порту 5173;
- npm run build - сборка production-версии в каталог dist;
- npm run preview - локальный просмотр собранной версии.

Те же инструкции продублированы в файле README.md в корне репозитория, как требует раздел отчётности задания.

## ЗАКЛЮЧЕНИЕ

В ходе работы разработано Vue-приложение, воспроизводящее лендинг IT-конференции из ЛР № 17. Реализованы три компонента обязательного набора и корневой компонент, состояние описано в опции data(), внешние библиотеки управления состоянием не применялись.

Освоен обмен данными между компонентами: свойства передаются вниз с указанием типов и обязательности, обратная связь идёт пользовательскими событиями. Показана разница между вычисляемым свойством и методом: expandedCount и toggleLabel пересчитываются только при изменении зависимостей. Директивы v-for, v-if, v-else-if, v-else и v-model применены по назначению, что позволило описать интерфейс декларативно, без прямых обращений к DOM.